

SOLID

SOLID es un acrónimo, representa 5 principios de la programación orientada a objetos. Son principios que indican buenas prácticas en el desarrollo orientado a objetos. Los principios que conforman SOLID son:

- **Single Responsibility:** Este principio significa Responsabilidad Única, y se refiere a que una clase debe de tener solo una responsabilidad. Es decir que no debe de realizar tareas más allá de su alcance.

Entonces una clase que representa, por ejemplo, un correo electrónico, no debe de encargarse de los protocolos de envío, la presentación del correo, ni de los requerimientos de la aplicación que se usan para enviar el correo. Lo que debería encargarse esta clase es que todos sus campos estén bien definidos, como que los correos de origen y destino tengan un formato de correo correcto, que los datos sean del tipo correcto, como que las fechas sean de tipo fecha.

¿En qué nos ayuda este principio? Nos ayuda a definir más claramente nuestras clases, a definir qué hace cada clase, cómo se relacionan y cómo se comunican las clases.

Veamos un ejemplo. Tenemos un programa de calculadora, que realiza las funciones esperadas, suma, resta, multiplicación, etc. Está definida de la siguiente forma:

```
class Calculadora {  
    ...  
    public String suma (int numero1, int numero2){  
        return "El resultado de la suma es: "+  
            (numero1 + numero2);  
    }  
    ...  
}
```

Esta calculadora funciona como lo esperaríamos, realiza la suma y nos imprime el resultado, ¿entonces cuál es el problema? El problema es que lo que está haciendo el problema es que regresa un String, que incluye el resultado en una oración, entonces no podemos hacer nada más con este resultado que mostrarlo en pantalla, no podemos compararlo con otros números, no podemos usarlo en otras operaciones.

Este es el problema que busca resolver este primer principio, La clase de Calculadora está realizando más de lo que debería, no le corresponde decidir cómo usar el resultado. Esta implementación funciona únicamente en este programa.

Una implementación mejor para este problema sería así:

```
class Calculadora {
    ...
    public int suma (int numero1, int numero2){
        return (numero1 + numero2);
    }
    ...
}

class Printer{
    ...
    public String presentarResultado(String operacion,
                                    int resultado){
        return "El resultado de la operacion: "+operacion+
               " es "+resultado;
    }
    ...
}
```

Tenemos una clase Calculadora, que es la que realiza las operaciones y una clase Printer, que formatea el resultado de las operaciones, ofreciendo estas dos clases permitimos que el usuario decida si quiere usar el resultado, o si prefiere tener un String para imprimir en pantalla.

Ojo: Una clase de este tipo nunca debe mostrar los resultados en pantalla, esto es porque no sabemos en donde se nos está llamando, puede que sea precisamente para presentar un resultado en pantalla, o puede que la clase se esté usando en un proceso aparte.

- **Open – Closed:** Este principio también se conoce como Abierto para extender, pero Cerrado para modificar, la importancia de este principio

se basa en el uso que le damos a las clases. Una clase en java es un código modular y reutilizable. Que una clase se pueda usar así es una gran ventaja, pero nos deja un gran problema, que, si los proyectos crecen mucho, no podemos estar seguros de donde está siendo usada nuestra clase. Si bien la usan en un proceso, o si otra clase depende de ella.

Por esto no debemos de modificar nuestras clases, las modificaciones pueden resultar en errores y comportamiento inesperado en procesos que la utilizan. Lo que sí podemos hacer es expandirlas, haciendo uso de la herencia, podemos crear clases más y más complejas que puedan ser usadas en una gran variedad de problemas.

Veamos un ejemplo, tenemos una clase que representa un Carro:

```
class Carro{
    String modelo = "Toyota Corolla 2010";
    String llantas = new String[4];
    int capacidadTanque = 10;
    int combustibleActual = 0;

    public String cargarCombustible(int cantidad){
        if(cantidad <= capacidadTanque){
            combustibleActual = cantidad;
            return "Carga exitosa";
        }else{
            return "error al cargar";
        }
    }
}
```

Esta clase representa, de manera muy simple un carro, pero ¿qué pasa si ahora queremos también utilizar carros eléctricos? Nuestra primera opción sería modificar la clase para que quede así:

```
class Carro{
    String modelo = "Toyota Corolla 2010";
    String llantas = new String[4];
    String tipo = "Gasolina";
    int capacidadTanque = 10;
    int combustibleActual = 0;

    public String cargarCombustible(int cantidad){
        if(tipo.equals("Gasolina")){
            if(cantidad <= capacidadTanque){
                combustibleActual = cantidad;
                return "Carga exitosa";
            }else{
                return "error al cargar";
            }
        }else{
            //cantidad sera el tiempo de carga
        }
    }
}
```

Agregamos el tipo de carro, si el tipo del motor es de gasolina, de diesel o si es un carro eléctrico.

Es funcionaria, pero recordemos el segundo principio ¿que pasa si ya estamos usando esta clase? Como este cambio se acaba de realizar, una vez que volvamos a compilar nuestro programa, las clases que dependen de nuestra clase Carro van a tener un comportamiento

inesperado, veamos el if, este primero revisa si el tipo es un carro de gasolina. Entonces cómo cambiamos la clase, y antes no tenía este tipo, nos va a dar un error, que no existe valor para esa propiedad.

Una mejor solución sería crear una clase nueva que represente los carros eléctricos.

```
class CarroElectrico extends Carro{
    @Override
    public String cargarCombustible(int cantidad){
        //Carga dependiendo del voltaje de carga y el tiempo
    }
}
```

Así podemos usar un carro eléctrico, y mantener el funcionamiento que ya tenemos del Carro.

Realizar esto también nos puede mostrar problemas en nuestro diseño. No siempre es lo mejor extender las clases. Si las diferencias entre las clases que tenemos y la que queremos definir son suficientes, es mejor que creamos una clase nueva.

Liskov Substitution La sustitución de Liskov es un principio que está relacionado con la jerarquía y herencia de las clases. Este principio nos dice que, si tenemos el uso de una clase Padre, se puede sustituir una clase Hija en su lugar sin afectar el comportamiento del programa.

Con esto nos referimos a que una clase hija debe de ser capaz de realizar las acciones de una clase padre.

Veamos un ejemplo, tenemos una clase de teléfono que, entre otras cosas, tiene una función de realizar llamadas.

```
class Telefono{  
    String numero;  
    public void realizarLlamada(String destino){  
        ...  
    }  
}
```

Y podríamos encontrar un uso de la siguiente forma:

```
Telefono t = new Telefono();  
t.realizarLlamada(numeroDestino);
```

Si definiéramos una clase hija, un Smartphone que fuera clase hija del teléfono, podremos realizar esa operación usando un Smartphone en lugar del Teléfono:

```
Smartphone s = new Smartphone();  
s.realizarLlamada(numeroDestino);
```

Realizar esto nos puede presentar otros problemas, como que el Smartphone requiere de un saldo, y solo puede realizar llamadas mientras este sea mayor a 0.

Esto nos puede llevar a replantear el diseño de nuestra clase padre.

- **Interface Segregation, o Separación de Interfaces:** se refiere al principio en el cual, una clase que usa una interfaz no debe de implementar métodos que no va a utilizar.

Pero antes de entrar a este principio recordemos ¿Que es una Interfaz?

Una interfaz es una colección de métodos que describe un comportamiento, esto es una forma de asignarle un comportamiento a una clase, sin hacerlo parte integral de la clase en sí. Pero también nos ofrece otro beneficio, nos permite crear componentes modulares que son fáciles de reemplazar. En lugar de depender de una clase en particular que se comporte como necesitamos, podemos depender de cualquier clase que utilice la interfaz.

Ahora veamos un ejemplo. Tenemos dispositivos multimedia, como lo son Celulares, Computadores, Televisores, entre otros. Queremos hacer que se puedan comunicar de manera inalámbrica y por eso definimos una interfaz.

```
public interface Comunicaciones{  
    public conectarWifi ();  
  
    public conectarBluetooth ();  
  
    public conectarCable();  
}
```

Esta interfaz tiene protocolos para comunicarse por wifi, por bluetooth o por cable.

Podemos pensar que es un buen diseño, después de todo es muy versátil y nos permite cubrir una gran cantidad de situaciones.

Sin embargo, no es correcto. El principio 4 nos dice que si una clase implementa una interfaz, dicha interfaz no debe de tener métodos que no le interesen a la clase.

Dicho de otra forma, todos los métodos que estén contenidos en una interfaz deben de ser importantes y útiles para el comportamiento que estamos describiendo.

Entonces ¿qué problema presenta esta interfaz? Que, al cubrir tantas opciones, estamos llenando la interfaz de métodos que no siempre van a ser útiles. Por ejemplo, un teléfono, lo más seguro es que no pueda enviar mensajes por cable, entonces no debería implementar estos métodos, pero en esta interfaz es requerido.

- **Dependency Inversión:** el último principio es otro que está relacionado con todos los anteriores, Inversión de Dependencias es un principio que trata sobre los módulos de un programa, más específicamente el desacoplamiento de estos. ¿A qué nos referimos con eso? a que una clase no debe de tener dependencias explícitas en otros módulos o clases, sino tratar de usar abstracciones siempre que sea posible.

Este principio se ve más claro con un ejemplo: Tenemos un programa que se encarga de enviar y recibir mensajes.

```
public class EnviarMensajes{
    EncriptadorBasico eb = new EncriptadorBasico();
    ModuloWifi mwifi = new ModuloWifi();
    public enviar(String mensaje){
        String mensajeEncriptado = eb.encriptar(mensaje);
        mwifi.enviar(mensajeEncriptado);
    }
}
```

¿Qué hace esta clase? Primero vemos que tiene un encriptador, luego un módulo wifi y finalmente un método que, haciendo uso del encriptador y el módulo wifi, envía mensajes encriptados.

La función de esta clase es simple y está clara, pero ¿qué problema tiene? Que la clase, que debería solo enviar mensajes, depende del funcionamiento del encriptador y el módulo wifi. Si alguno de estos cambia su funcionamiento, nuestra clase también va a tener que cambiar.

Inversión de Dependencia se refiere a este problema, donde una clase, en este caso una clase de mensajería depende de otra, en este caso una clase de encriptación.

La solución, como nos dice este principio, es depender de las abstracciones y no de las clases, entonces lo que hay que hacer es usar una interfaz.

```
public interface Encriptar{
    encriptar();
}

public class EnviarMensajes{
    Encriptar encriptador;
    ModuloWifi mwifi = new ModuloWifi();
    public colocarEncriptador(Encriptar e){
        this.encriptador = e;
    }

    public enviar(String mensaje){
        String mensajeEncriptado =
encriptador.encriptar(mensaje);
        mwifi.enviar(mensajeEncriptado);
    }
}
```

Con esta solución, no estamos dependiendo de una clase en concreto, sino en la abstracción de un encriptador, no de un encriptador en particular, sino del comportamiento de este.

De esta forma nuestra clase no está dependiendo del comportamiento de otra clase.

Los principios que nos da SOLID son una buena guía para el diseño de programas, pero como todas las guías no siempre es posible cumplirlas en su totalidad.